approach will be naturally assumed. However, for most of our data structures we will only need arrays and containers that hold elements of the same type, and unless it's stated differently, you can safely assume this will be the case: just be aware that handling arrays with heterogeneous types is possible but requires extra effort and care.

As for the nomenclature, to access an array element we use square brackets, like most programming languages do:

`A[i]` denotes the value of the i-th element of array `A`, and similarly to assign a value to that element, we use the regular syntax we have shown for variables, something like

`A[j] = b` (that assigns the value of the variable b to the j-th element of array `A`).

Whenever it's not stated differently, we will assume arrays use 0-based indexing, so the first element in array `A` will be `A[0]`, the second one `A[1]`, and so on.

We will also, occasionally, have to perform certain operations on arrays (and containers in general), for example we might need to find the maximum value in an array of numbers, or the longest word in an array of strings, or yet the sum of values in a numeric array.

For these array-wide operations, we will use something closer to mathematical notation, for the sake of synthesis. For instance, given an array `A`:

- `max{A}` stands for the maximum value of `A`
- `sum{A}` is the sum of all element of `A`